

Day 02: アプリを動かそう

今日のゴール

GitHubからアプリをダウンロードして、あなたのパソコンで実際に動かせるようになります。ブラウザで `http://localhost:3000` にアクセスすると、task-appの初期画面が表示されます。

【スクリーンショット: localhost:3000にアクセスした初期画面】

なぜこれを作るのか？

プログラミングを学ぶ上で、「動くものを実際に触る」ことが最も重要です。教科書を読むだけでは理解できないことも、実際に動かしてみれば「なるほど！」と腑に落ちます。

例え話: 料理の作り方を本で読んだだけでは、実際の味はわかりません。実際に調理して、味見をして初めて「この味か！」とわかります。プログラミングも同じで、コードを書いて動かしてこそ、本当の理解が得られます。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	GitHubからコードをダウンロード	5分
Step 2	Dockerでデータベースを起動	10分
Step 3	必要なパッケージをインストール	10分
Step 4	アプリを起動して動作確認	10分

合計時間: 約35分

Step 1: GitHubからコードをダウンロード（5分）

ゴール: task-appのコードを自分のパソコンにダウンロードします。

初心者向け解説: GitHubは、プログラムのコードを保管・共有するためのサービスです。 `git clone` というコマンドを使うと、GitHubに保存されているコードを自分のパソコンにコピーできます。まるで、図書館から本を借りるようなイメージです。

実装:

```
# filepath: ターミナル（コマンドラインで実行）
$ cd ~
$ git clone https://github.com/your-username/task-app.git
$ cd task-app
```

コード解説:

コマンド	意味	例え
<code>cd ~</code>	ホームディレクトリ（自分のフォルダ）に移動	自分の部屋に戻る
<code>git clone [URL]</code>	GitHubからコードをダウンロード	図書館から本を借りる
<code>cd task-app</code>	task-appフォルダに移動	本を読むために本棚を開く

確認ポイント:

1. ターミナルで `ls` と入力してEnterを押す
2. `src`, `prisma`, `package.json` などのフォルダやファイルが表示される
3. これでコードのダウンロードが完了です

【スクリーンショット: ターミナルでlsを実行した画面】

学んだこと: `git clone` コマンドを使って、GitHubからコードをダウンロードできるようになりました。

Step 2: Dockerでデータベースを起動（10分）

ゴール: PostgreSQLデータベースをDockerで起動します。

初心者向け解説: Dockerは、アプリケーションを「箱」に入れて動かす技術です。データベース（PostgreSQL）も、Dockerという箱の中で動かします。箱の中で動かすことで、あなたのパソコンの環境を汚さず、いつでも同じ状態で起動できます。

実装:

```
# filepath: ターミナル (task-appフォルダ内で実行)
$ docker compose up -d
```

コード解説:

コマンド	意味	例え
<code>docker compose up</code>	Dockerコンテナを起動	箱を開けて中のものを動かす
<code>-d</code>	バックグラウンドで実行 (detached mode)	裏で動かし続ける

確認ポイント:

1. ターミナルに `Creating taskapp-postgres ... done` のようなメッセージが表示される
2. Docker Desktopを開くと、`taskapp-postgres` が緑色 (Running) で表示される
3. これでデータベースが起動しました

【スクリーンショット: Docker Desktopでtaskapp-postgresがRunning状態の画面】

学んだこと: `docker compose up -d` コマンドで、データベースをバックグラウンドで起動できるようになりました。

Step 3: 必要なパッケージをインストール (10分)

ゴール: アプリに必要なライブラリ (npm パッケージ) をインストールします。

初心者向け解説: アプリは、多くの外部ライブラリ (部品) に依存しています。これらをダウンロードしないと、アプリは動きません。`npm install` コマンドを使うと、`package.json` に書かれたすべてのライブラリを一括でインストールできます。

実装:

```
# filepath: ターミナル (task-app フォルダ内で実行)
$ npm install
```

コード解説:

コマンド	意味	例え
<code>npm install</code>	<code>package.json</code> に書かれた全パッケージをインストール	レシピに書かれた食材を全て買いそろえる

確認ポイント:

1. ターミナルに `added XXX packages` のようなメッセージが表示される
2. `node_modules` フォルダが作成される
3. これで必要なライブラリがすべてインストールされました

【スクリーンショット: npm install の実行結果】

学んだこと: `npm install` コマンドで、アプリに必要なすべてのライブラリを一括インストールできるようになりました。

Step 4: アプリを起動して動作確認（10分）

ゴール: Next.jsの開発サーバーを起動して、ブラウザでアプリを確認します。

初心者向け解説: `npm run dev` コマンドを実行すると、Next.jsの開発サーバーが起動します。開発サーバーが起動すると、ブラウザで `http://localhost:3000` にアクセスすることで、アプリを確認できます。コードを変更すると、ブラウザが自動でリロードされるので、すぐに結果を確認できます。

実装:

```
# filepath: ターミナル (task-appフォルダ内で実行)
$ npm run dev
```

コード解説:

コマンド	意味	例え
<code>npm run dev</code>	開発サーバーを起動	コンロに火をつける

確認ポイント:

1. ターミナルに `Local: http://localhost:3000` と表示される
2. ブラウザで `http://localhost:3000` にアクセスする
3. task-appの初期画面が表示される
4. これでアプリが正常に起動しています

【スクリーンショット: ブラウザでlocalhost:3000にアクセスした画面】

学んだこと: `npm run dev` コマンドで、開発サーバーを起動し、ブラウザでアプリを確認できるようになりました。

今日のまとめ

- ☐ `git clone` でGitHubからコードをダウンロードできた
- ☐ `docker compose up -d` でデータベースを起動できた
- ☐ `npm install` で必要なライブラリをインストールできた
- ☐ `npm run dev` でアプリを起動できた
- ☐ ブラウザで `http://localhost:3000` にアクセスできた

⚠ つまづきポイント

エラー/問題	原因	解決方法
<code>git clone</code> が失敗する	Gitがインストールされていない	公式サイトからGitをインストールしてください
<code>docker compose up</code> が失敗する	Docker Desktopが起動していない	Docker Desktopを起動してください
<code>npm install</code> が失敗する	Node.jsがインストールされていない	公式サイトからNode.jsをインストールしてください
<code>localhost:3000</code> にアクセスできない	開発サーバーが起動していない	<code>npm run dev</code> を実行してください

次回予告

Day 3では、今日動かしたアプリをGitHubに保存する方法を学びます。コードをGitHubに保存することで、バックアップを取ったり、他の人と共有したりできるようになります。